

CS 455 / CS 555 - Project Proposal

Large Language Models - Spring 2025/2026

Section 1 - Group Members

Mehmet_Onur_Sönmez, 32278

Efe_Sözer, 31012

Mitat_Mehmet_Erşeker, 31258

Section 2 - Track

Our track: CS 455 Project

Section 3 - Project Proposal

3.1 Project Title

Reliability-Aware RAG for Turkish Legal Question Answering

3.2 Project Category

A - Application / System Building. The project fits this category because we will build a working end-to-end Turkish legal question-answering application with retrieval, answer generation, source display, and a reliability-checking layer. The system is not intended to be a professional legal advisor; it is an educational and informational RAG system that makes grounding and uncertainty visible to the user.

3.3 Problem Definition

Turkish legal information is difficult for non-expert users because it is formal, lengthy, and often distributed across question-answer pages, legal explanations, and related documents. Our system will take a Turkish legal question as input and return a short, understandable Turkish answer based on retrieved legal QA/explanation passages. It will also show the retrieved passages and a reliability analysis that warns the user when the answer is only partially supported, unsupported, ambiguous, or risky as legal advice. Success means that the RAG-based system retrieves relevant passages, produces more grounded answers than an LLM-only baseline, and makes reliability problems visible through a verifier layer.

3.4 Motivation

Large language models can explain complicated topics in simple language, but they can also produce fluent unsupported answers. This is especially risky in legal domains, where a wrong answer may mislead a user into making an important decision. RAG can reduce this risk by forcing answer generation to use retrieved source passages, but retrieval can fail and the generator can still hallucinate. For this reason, we are interested in building a legal QA assistant that does not only answer the question, but also shows its retrieved evidence and adds a verifier-style warning mechanism for unsupported or risky claims.

3.5 Approach

We will build the system in four stages: data preparation, retrieval, answer generation, and reliability checking. First, we will load Turkish legal QA datasets, clean duplicated or unusable records, and convert each record into a searchable legal QA/explanation passage. We will embed passages using a Turkish-compatible multilingual sentence embedding model such as `intfloat/multilingual-e5-base` or `sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2`, store vectors in FAISS or Chroma, and retrieve the top-k passages for each user question. If time allows, we will add a multilingual reranker to improve retrieval order.

For answer generation, the retrieved passages and user question will be passed to a feasible open-source instruction-tuned model selected based on Colab/Kaggle constraints. The prompt will instruct the model to answer only from retrieved context, cite source passages, avoid presenting itself as a lawyer, and refuse or qualify the answer when context is insufficient. After generation, a verifier prompt will inspect the question, retrieved passages, and answer, then return structured labels such as supported, partially supported, unsupported, insufficient context, and legal advice risk. We will compare three versions: an LLM-only baseline, a RAG baseline, and RAG + verifier. For the demo, we will implement a Streamlit or Gradio interface showing the question, generated answer, retrieved passages, reliability labels, and a clear educational-use disclaimer that encourages users to consult a qualified lawyer for case-specific decisions.

3.6 Data

Our main dataset will be `OrionCAF/turkish_law_qa_dataset` from Hugging Face, which contains approximately 18,305 Turkish legal question-answer pairs and is listed with an Apache 2.0 license. We will use the answer or explanation fields as searchable passages and keep metadata such as question text, dataset name, and record ID where available. As an optional extension, we may use `alibayram/hukuk_soru_cevap` only if its license and usage conditions are compatible with course project use. We may also inspect `Turkish-NLI/legal_nli_TR_V1` as inspiration for entailment-style labeling, but we will not depend on it because its task structure may not directly match our QA support-checking setup. We will not use private user data, and we will document dataset sources, preprocessing decisions, snapshot dates, and limitations in the final report.

3.7 Evaluation Plan

We will evaluate retrieval quality, answer quality, and verifier quality. For retrieval, we will prepare around 40-50 Turkish legal questions using held-out dataset records and manually written ambiguous, unsupported, and legal-advice-risk examples; where possible, we will use gold passage IDs or manually annotated relevant passages and report Recall@3, Recall@5, and MRR. For answer quality, we will manually compare LLM-only, RAG, and RAG + verifier outputs using a fixed rubric covering correctness, source faithfulness, clarity, refusal behavior, and avoidance of unsafe legal advice. For verifier quality, we will manually label a sample of answers as supported, partially supported, unsupported, or risky, then compare those labels with the verifier output using simple precision/recall-style statistics where feasible and qualitative error analysis. We will also compare at least two ablations, such as top-k values, strict vs permissive prompts, or retrieval with vs without reranking.

3.8 Timeline

Week / Dates	Planned Milestones / Deliverables
Week 1 (now - May 10)	Finalize topic, revise proposal, inspect dataset schemas, define exact scope, and choose the first embedding/vector database setup.
Week 2 (May 11 - May 17)	Load and clean the dataset, prepare searchable passages, create embeddings, build the first FAISS or Chroma index, and create an initial evaluation set.
Week 3 (May 18 - May 24)	Build the first RAG pipeline, test top-k retrieval, implement the generation prompt, and run preliminary retrieval metrics.
Week 4 (May 25 - May 31)	Add the verifier layer, define reliability labels, compare LLM-only / RAG / RAG + verifier, and start manual error analysis.
Week 5 (June 1 - June 7)	Finish evaluation, prepare the demo interface, clean code and README, write the final report, and prepare demo examples.

3.9 Risks and Contingency Plan

- Risk: Dataset grounding may be weaker than expected because the main corpus contains QA-style explanations rather than official statute articles in every case. Plan B: We will describe the corpus honestly as legal QA/explanation passages, optionally add public statute snippets if time allows, and evaluate whether answers are supported by the retrieved corpus rather than claiming official legal completeness.
- Risk: The system may be interpreted as real legal advice. Plan B: The UI and prompts will clearly frame the system as educational/informational, include a disclaimer, encourage users to consult a qualified lawyer for case-specific decisions, and use the verifier to flag action-oriented or case-specific recommendations as legal-advice risk.
- Risk: Turkish model quality or compute limits may reduce generation quality. Plan B: We will not depend on full fine-tuning; we will use RAG, prompting, small or quantized open-source models, and manual evaluation. If generation quality is weak, we will focus the final contribution on retrieval quality, stricter source-grounded prompts, source display, and reliability analysis.

3.10 Compute Resources

- [x] Google Colab (free tier)
- [x] Kaggle Notebooks
- [] Free credits from Google / other platforms
- [] Paid APIs - OpenAI / Anthropic / other
- [x] Local hardware
- [] Other

We will mainly use Google Colab Free and Kaggle Notebooks for preprocessing, embedding generation, retrieval tests, and model inference. This compute plan matches the project scope because we do not require full fine-tuning of a large model. FAISS or Chroma indexing and evaluation can be run with modest resources, and model selection will be kept within free Colab/Kaggle constraints.

3.11 Expected Outcomes

By June 7, we expect to deliver a working Turkish legal QA RAG assistant with a verifier layer. The demo will allow a user to type a Turkish legal question and see the generated answer, retrieved passages, and reliability labels. The final report will include system design, dataset preprocessing, retrieval setup, prompts, evaluation results, error analysis, limitations, and future work. The code submission will include a clear README so the system can be installed and run again.

Section 4 - References

Ali Bayram. Turkce Hukuki Soru-Cevap Veri Seti. https://huggingface.co/datasets/alibayram/hukuk_soru_cevap
Chroma Documentation. <https://docs.trychroma.com/>
FAISS: A library for efficient similarity search. <https://github.com/facebookresearch/faiss>

Gradio Documentation. <https://www.gradio.app/docs>

Lewis, P. et al. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. <https://arxiv.org/abs/2005.11401>

OrionCAF. Turkish Law QA Dataset. https://huggingface.co/datasets/OrionCAF/turkish_law_qa_dataset

Sentence Transformers Documentation. <https://www.sbert.net/>

Turkish-NLI. legal_nli_TR_V1 Dataset. https://huggingface.co/datasets/Turkish-NLI/legal_nli_TR_V1

Wang, L. et al. (2024). Multilingual E5 Text Embeddings: A Technical Report. <https://arxiv.org/abs/2402.05672>